PROG2002 - Web Development II

Assignment 3: A Complete Dynamic Website

Student ID: 24831941 Last Name: Bao First Name: Jie

Team: Jie Bao (24831941) and Chengyu Gao (24832054) - 2 members

Title of the project: Giving Coast - Charity Events on the Gold Coast

Live client site: https://24831941.it.scu.edu.au/

Live admin site: https://24831941.it.scu.edu.au/admin/

GitHub repository: https://github.com/BaoJie001/PROG2002-A3 (private, marker invited)

1. Introduction / Motivation

Assessment 2 produced a working dynamic website. Three pages - Home, Search and Event details - read charity events out of a MySQL database through a small Express API and rendered them in the browser. It was a real client/server application, but it could only read. A member of the public who saw an event they liked had no way to book a place, and a staff member who needed to correct a venue or cancel a washed-out fun run had to open phpMyAdmin and edit a row by hand.

Assessment 3 closes both of those gaps. The public site gains a registration page and an event page that shows who has already booked. The staff site is new: it lists every event regardless of status and supports the full create, read, update and delete cycle. The database gains a registrations table, the API moves from read-only to full CRUD, and both websites plus the API are deployed to SCU cPanel so they can be reached from the internet rather than only from localhost.

We also went past the minimum in three places, because the marking rubric rewards them and because they made the project more interesting to build. Both websites are Angular 22 applications written in TypeScript rather than plain DOM code; the event page shows a live weather forecast for the venue, fetched from the Open-Meteo API;

and there is a Postman collection with assertions so that every endpoint can be re-checked in one click. Section 8 records that extra work separately so it can be marked as such.

2. Problem Statement

The case study is unchanged from Assessment 2: four small Gold Coast charities - a running club, a community kitchen, a music school and a wildlife shelter - run a steady stream of fundraisers and need one place to publish them. What changes is that the site now has to accept data as well as display it, and that it has to survive being deployed on shared hosting. That raises three problems that are not obvious from the requirement list.

Modelling a booking when there is no user account. The brief says there is no USER table, and it also says that one person may register for many events but only once per event. Both have to be true at the same time. If a person's only identity is their email address, then the uniqueness rule is really a rule about the pair (event, email). It has to be enforced by the database rather than by a check in JavaScript, because two requests that arrive within the same millisecond can both pass a JavaScript check and both insert a row.

Deleting an event without breaking the register. The rule 'an event can only be deleted if it has received no registrations' is a statement about data integrity, not about the interface. It has to hold even when somebody calls the API directly with curl, and it has to hold when a booking arrives at the same moment as a delete. Hiding the Delete button on the admin screen is a courtesy to the user, not a guarantee to the database.

One build, three addresses. On cPanel the public site is served from the root of the domain, the admin site from /admin/, and the API from a Node application mounted at /api. A single-page application normally assumes it owns the root of the URL space: it loads its scripts from /main.js and it takes over every path so that its own router can decide what to show. Both assumptions are false here, and the deployment has to correct them.

3. Solution

The site keeps the three-layer shape from Assessment 2 and adds a second client to it. Nothing in the browser ever talks to MySQL; everything goes through the API.

Table 1 - The four parts of the system.

Layer	Technology	Responsibility
Database	MySQL 8.0	charityevents_db: organisations, categories, events, registrations. Owns the integrity rules (foreign keys, unique key, CHECK constraints).
Server	Node.js 20+ / Express 4.21 / mysql2 3.11	The only process with database credentials. Validates input, runs parameterised SQL, maps results to JSON, decides status codes, and proxies the weather API.
Public client	Angular 22.2 / TypeScript 6	Home, Search, Event details and Register pages. Calls the API with HttpClient and renders the result with signals.
Staff client	Angular 22.2 / TypeScript 6	Event list, new event, edit event. Same API, different endpoints (it may also write).

A page request always follows the same path. The Angular router matches a URL and creates a component; the component asks a service for data; the service calls the API with HttpClient and returns a Promise; Express receives the request, validates the parameters and runs one parameterised query through the mysql2 connection pool; MySQL returns rows; Express serialises them to JSON; the service resolves the Promise; the component writes the result into a signal; and the template, which is already bound to that signal, re-renders. No part of that chain knows about the layer after it, which is why the same API can serve two different front ends and why we could replace the A2 front end without touching a single SQL statement.

Concretely, a booking works like this. On /event/5 the visitor presses Register. The button links to /register?eventId=5 - a query string, not a JavaScript variable, so the page can be refreshed, bookmarked or shared and still knows which event it is for. The register page reads eventId from the URL, fetches the event so it can show the name, date, venue and price, and fills in the ticket total live as the visitor changes the quantity. On submit it POSTs to /api/registrations. The API validates the body, checks that the event exists and is still accepting bookings, inserts the row, and answers 201 with the new record. The page shows a confirmation and a link back to the event, and the event page now lists the new booking first because it sorts by registration date descending.

4. Data Schema

Two changes were needed: a new table to hold bookings, and two columns on events so that a weather forecast can be requested for the right place. Everything else from Assessment 2 was left alone, because it already modelled the case study correctly.

Table 2 - The registrations table.

Column	Type	Why it exists
registration_id	INT AUTO_INCREMENT PK	Surrogate key. The natural key would be (event_id, email), which is two wide columns; a single integer is cheaper to index and easier to reference.
event_id	INT NOT NULL, FK -> events	Which event was booked. ON UPDATE CASCADE, ON DELETE RESTRICT.
first_name / last_name	VARCHAR(80) NOT NULL	Stored separately from email so the staff screen can sort and greet by name, and so a shared family address still reads correctly.
email	VARCHAR(150) NOT NULL	Contact detail, and part of the uniqueness rule.
phone	VARCHAR(30) NOT NULL	The charities ring people when a event is cancelled - an email sent the night before is often missed.
tickets	INT NOT NULL DEFAULT 1	How many places were bought. CHECK (tickets > 0 AND tickets <= 50).
registration_date	DATE NOT NULL	The day the visitor says they booked. This is a business value, not a timestamp, which is why it is a DATE.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	When the row was actually written. Kept separate from registration_date so the two can never be confused when auditing.
notes	VARCHAR(255) NULL	Dietary requirements, accessibility needs, 'arriving late'.

The constraints are where the design actually lives, so they are worth spelling out:

```
CONSTRAINT fk_registration_event
    FOREIGN KEY (event_id) REFERENCES events (event_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,

CONSTRAINT uq_registration_event_email
    UNIQUE (event_id, email),

CONSTRAINT chk_tickets_positive CHECK (tickets > 0),
CONSTRAINT chk_tickets_sane     CHECK (tickets <= 50),
CONSTRAINT chk_email_shape      CHECK (email LIKE '%_@%_._%')
```

The UNIQUE key is the answer to the first problem in Section 2: 'one person, one booking per event' becomes a property of the table, so it holds no matter how many requests arrive at once, and the API only has to translate the resulting duplicate-key error into a friendly 409. ON DELETE RESTRICT is the answer to the second problem: MySQL itself refuses to delete an event that still has bookings, so the rule

survives a careless DELETE typed into phpMyAdmin. The API checks first and returns a helpful message, but the constraint is the thing that guarantees it.

Two values are deliberately not stored. The total cost of a booking is tickets multiplied by the event's ticket_price, and the number of places still available is the event's capacity minus the sum of tickets sold. Both are derived when they are needed. Storing them would create a second copy of the truth that could drift out of step the first time a price changed.

On the events table, latitude and longitude were added as DECIMAL(9,6), which is accurate to about 11 centimetres - far more than a weather grid cell needs, and enough that the same columns could later be used to draw a map. Nullable, because not every venue has been geocoded yet, and the weather panel has to cope with that.

The sample data was extended to 16 registrations spread across five events, comfortably over the required minimum of 10, so that the 'who is going' list on the event page and the registration list on the admin edit screen both have something realistic to show. The whole schema and dataset still export to a single SQL file, database/charityevents-db.sql, as the brief requires.

5. API Design

The API exposes eleven endpoints plus a health check. They are grouped by resource rather than by page, so the shape of the URL says what is being acted on:

Table 3 - The endpoints. Every one of them is called by at least one page.

Method	Path	Purpose	Success	Errors
GET	/api/events	Upcoming events, optionally filtered by date, location, category and free-text search	200	400
GET	/api/events/past	Past events, for the 'previously' section of the home page	200	-
GET	/api/events/:id	One event plus its complete registration list, newest booking first	200	400, 404
POST	/api/events	Create an event (admin)	201	400
PUT	/api/events/:id	Replace an event's details (admin)	200	400, 404
DELETE	/api/events/:id	Delete an event, blocked when bookings exist	200	400, 404, 409
GET	/api/registrations	All bookings, or those for one event via ?event_id=	200	400
POST	/api/registrations	Create a booking (public site)	201	400, 404, 409